

Unit-VI

Branch and Bound: General method,
Applications - Travelling sales person problem,
0/1 knapsack problem - LC Branch and Bound
solution, FIFO Branch and Bound solution.
Introduction to NP-Hard and NP-Complete
problems: Basic concepts of non deterministic
algorithms, Definitions of NP-Hard and NP-
Complete classes.
Modular Arithmetic

Branch and Bound

- Backtracking is effective for *subset* or *permutation* problems.
- Backtracking is not good for *optimization* problems.
- This drawback is rectified by using *Branch And Bound* technique.
- *Branch And Bound is* applicable for only *optimization* problems.
- Branch and Bound also uses *bounding function* similar to backtracking.

Terminology of tree organization

Problem state :-

Each node in the tree is a problem state.

Ex :-

① ② ①⑧ *and so on*

State Space :-

All paths from the root to problem states define the *state space* of the problem.

Solution States :-

These are those problem states S for which the path from the root to S define a tuple in the solution space.

Ex:-

(5) (7) (10) (12) (15) (17) (21) and so on

Solution Space :-

All paths from the root to *solution states* define the *solution space* of the problem.

- A node which has been generated and all of whose children have not yet been generated is called a *live node*.
- The *live node* whose children are currently being generated is called *E-node* (*Expanding node*).
- A *dead node* is a node which can not to be expanded further or all of whose children have been generated.

Branch And Bound

Nodes will be expanded in three ways.

- FIFO branch and bound -- queue
- LIFO branch and bound -- stack
- *Least-Cost (or max priority) branch and bound -- priority queue.*
 - *Least-cost* branch and bound directs the search to the parts which most likely to contain the answer.

Bounding:

- The bounding functions are used to avoid the generation of sub trees that do not contain the answer nodes. In bounding *upper* and *lower* bounds are generated at each node. $\wedge$
- A cost function $c(.)$ such that $c(x) \leq c(x)$ is used to provide the *lower* bounds on solutions obtainable from any node x . $\wedge$
- If *upper* is an upper bound, then all live nodes x with $c(x) > upper$ may be killed as those parts do not contain answer nodes.
- *upper* is updated whenever a child is generated.

0/1 Knapsack Problem

- Generally Branch and Bound will minimize the objective function.
- The 0/1 knapsack problem is a maximization problem.
- This difficulty can be avoided by replacing the objective function $\sum p_i x_i$ by $-\sum p_i x_i$.

Note:

^

1. All live nodes with $c(x) > \text{upper}$ can be killed when they are about to become E-nodes .

LC Branch and Bound

- Steps to be followed in LCBB

Step 1: Compute $\hat{c}(\cdot)$ and $u(\cdot)$ for a generated node, then if $\hat{c} > \text{upper}$ (initially $\text{upper} = u(\text{root node})$) immediately kill the node otherwise add the node to the list of live nodes.

Step 2: Update $\text{upper} = u(\cdot)$, if upper is greater than $u(\cdot)$.

Step 3: The Least Cost node will be the next E-node.

Step 4: Whenever a node about to become an E-node, check if $\hat{c}(\cdot)$ is greater than upper, if yes kill the node otherwise generate its children. *Also kill the nodes which are not feasible.*

Step 5: Continue step 1 to 4 till all the live nodes are expanded.

Step 6 : Finally the highest U value (absolute value) node will be the answer node. Trace the path in backward direction from answer node to root node for solution subset.

LC Branch and Bound Solution

Upper number = $\hat{C}$

Lower number = u

EX:- $n=4$, $(p_1, p_2, p_3, p_4) = (10, 10, 12, 18)$

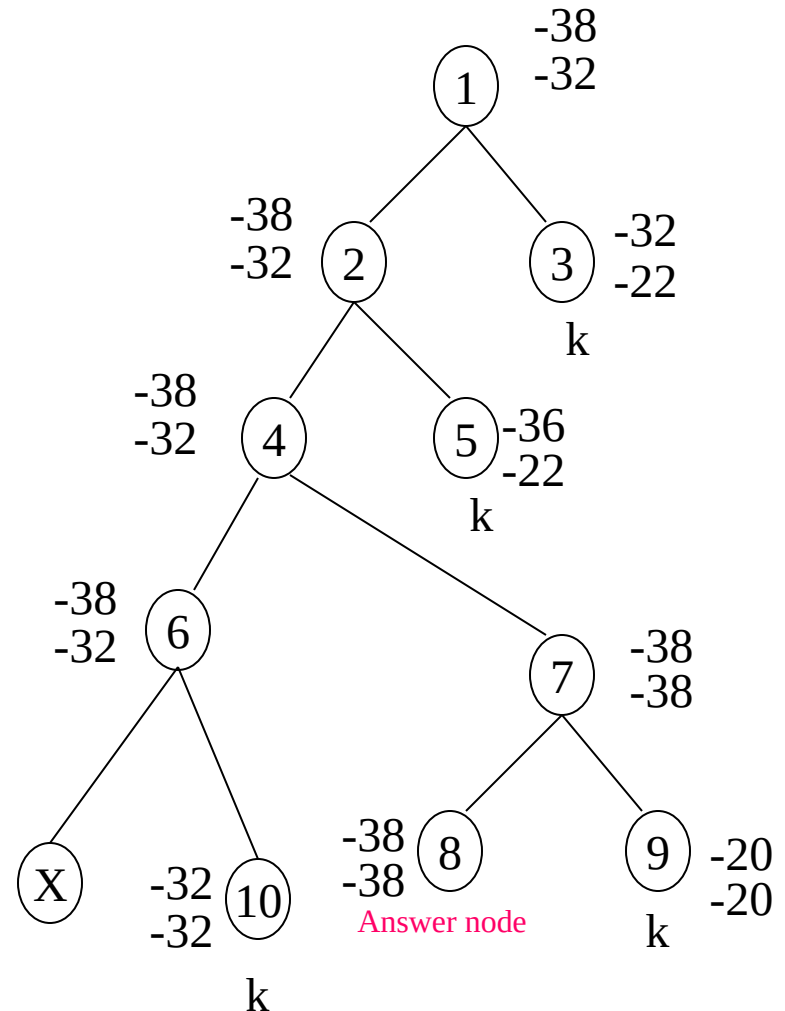
$(w_1, w_2, w_3, w_4) = (2, 4, 6, 9)$, $m=15$

Process: The calculation of U and $\hat{C}$ is as follows.

$U(1)$ - Scan through the objects from left to right and put into the knapsack until the first object that does not fit is encountered.

$\hat{C}(1)$ - Similar to $U(1)$ except that it also considers a fraction of the first object that does not fit the knapsack.

Continue this process until an answer node is found.



- Let us trace the working of an LC Branch and Bound using $\hat{c}(\cdot)$ and $u(\cdot)$ as defined previously.
- The search begins with the root as the E-node. For this node, we have $\hat{c}(1) = -38$ and $u(1) = -32$. Initially, $\text{upper} = u(1) = -32$.
- Since node 1 is not a solution node, its two children, node 2 & 3, are generated. The cost $\hat{c}(2) = -38$, $\hat{c}(3) = -32$, $u(2) = -32$ and $u(3) = -22$. Since both the nodes $\hat{c}()$'s are not greater than the upper, nodes are put onto the list of live nodes.
- Node 2 (least cost) is the next E-node. It is expanded and nodes 4 and 5 are generated. Both the nodes get added to the list of live nodes as their $\hat{c}()$'s values are not greater than the upper.
- Node 4 will be the next E-node, its children 6 and 7 are generated. Assuming node 6 is generated first, it is added to the list of live nodes. Next node 7 joins this list and upper is updated to -38.
- The next E-node will be one of the nodes 6 and 7 (least cost node). Let us assume it is node 7. Its two children are nodes 8 and 9. Node 8 is put onto the live nodes list. Node 9 has $\hat{c}(9) > \text{upper}$ and is killed immediately.

contd..

- Node 8 can not be expanded further as there are only 4 items.
- Node 6 is the next E-node, its left child is not a feasible node. Node 10 is generated and killed immediately as its $\hat{c}(10) > \text{Upper}$. Similarly node 3 and 5 are killed as their $\hat{c}()$'s values are greater than upper. Node 8 is the answer node (highest absolute $u()$ value).
- **Maximum profit is 38.**
- **Solution (x_1, x_2, x_3, x_4) is $(1, 1, 0, 1)$.**

FIFO Branch and Bound

- Steps to be followed in FIFOBB

Step 1: Compute $c(.)$ and $u(.)$ for a generated node, then if $c > \text{upper}$ (initially $\text{upper} = u(\text{root node})$) immediately kill the node otherwise add the node to the list of live nodes.

Step 2: Update $\text{upper} = u(.)$, if upper is greater than $u(.)$.

Step 3: The next node in the queue (FIFO) will be the next E-node.

Step 4: Whenever a node about to become an E-node, check if $c(.)$ is greater than upper , if yes kill the node otherwise generate its children. Also kill the nodes which are not feasible.

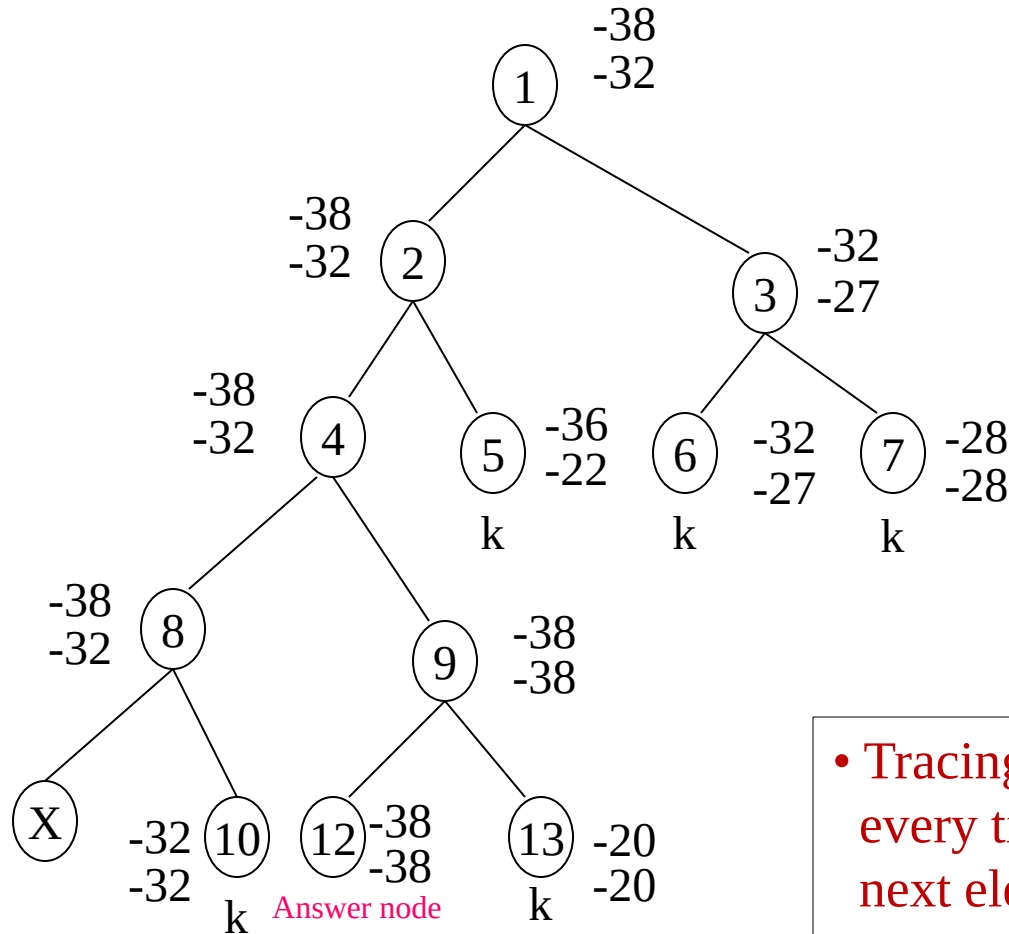
Step 5: Continue step 1 to 4 till all the live nodes are expanded.

Step 6 : Finally the highest U value (absolute value) node will be the answer node. Trace the path in backward direction from answer node to root node for solution subset.

FIFO Branch and Bound

Upper number = $\hat{c}$

Lower number = u



- Tracing same as LCBB, except that every time the E-node will be the next element in the queue .
- Max Profit is 38.
- Solution is (1,1,0,1).

1. Draw the portion of the state space tree generated by LCBB and FIFIBB for the following knapsack problem.

a) $n=5$, $(p_1, p_2, p_3, p_4, p_5) = (10, 15, 6, 8, 4)$,
 $(w_1, w_2, w_3, w_4, w_5) = (4, 6, 3, 4, 2)$ and $m=12$

b) $n=5$, $(p_1, p_2, p_3, p_4, p_5) = (w_1, w_2, w_3, w_4, w_5) =$
 $(4, 4, 5, 8, 9)$ and $m=15$.

Traveling Sale Person Problem:

By using dynamic programming algorithm we can solve the problem with time complexity of $O(n^2 2^n)$ for worst case. This can be solved by branch and bound technique using efficient bounding function. The time complexity of traveling sale person problem using LC branch and bound is $O(n^2 2^n)$ which shows that there is no change or reduction of complexity than previous method.

Problem Statement : If there are n cities and cost of traveling from any city to any other city is given. Then we have to obtain the shortest round-trip such that each city is visited exactly once and then returning to starting city, completes the tour.

Procedure for solving traveling sale person problem:

1. Reduce the given cost matrix. A matrix is reduced if every row and column is reduced. A row (column) is said to be reduced if it contain at least one zero and all-remaining entries are non-negative. This can be done as follows:
 - a) *Row reduction:* Take the minimum element from first row, subtract it from all elements of first row, next take minimum element from the second row and subtract it from second row. Similarly apply the same procedure for all rows.
 - b) Find the sum of elements, which were subtracted from rows.
 - c) Apply column reductions for the matrix obtained after row reduction.

Column reduction: Take the minimum element from first column, subtract it from all elements of first column, next take minimum element from the second column and subtract it from second column. Similarly apply the same procedure for all columns.
 - d) Find the sum of elements, which were subtracted from columns.
 - e) Obtain the cumulative sum of row wise reduction and column wise reduction.

Cumulative reduced sum = Row wise reduction sum + column wise reduction sum.

Associate the cumulative reduced sum to the starting state as lower bound and ∞ as upper bound.
2. Calculate the reduced cost matrix for every node R. Let A is the reduced cost matrix for node R. Let S be a child of R such that the tree edge (R, S) corresponds to including edge $\langle i, j \rangle$ in the tour. If S is not a leaf node, then the reduced cost matrix for S may be obtained as follows:
 - a) Change all entries in row i and column j of A to ∞ .
 - b) Set A (j, 1) to ∞ .
 - c) Reduce all rows and columns in the resulting matrix except for rows and column containing only ∞ . Let r is the total amount subtracted to reduce the matrix.
 - c) Find $\bar{c}(S) = \bar{c}(R) + A(i, j) + r$, where 'r' is the total amount subtracted to reduce the matrix, $\bar{c}(R)$ indicates the lower bound of the i^{th} node in (i, j) path and $\bar{c}(S)$ is called the cost function.
3. Repeat step 2 until all nodes are visited.

Example:

Find the LC branch and bound solution for the traveling sale person problem whose cost matrix is as follows:

The cost matrix is
$$\begin{bmatrix} \infty & 20 & 30 & 10 & 11 \\ 15 & \infty & 16 & 4 & 2 \\ 3 & 5 & \infty & 2 & 4 \\ 19 & 6 & 18 & \infty & 3 \\ 16 & 4 & 7 & 16 & \infty \end{bmatrix}$$

Step 1: Find the reduced cost matrix.

Apply row reduction method:

*Deduct 10 (which is the minimum) from all values in the 1st row.
Deduct 2 (which is the minimum) from all values in the 2nd row.
Deduct 2 (which is the minimum) from all values in the 3rd row.
Deduct 3 (which is the minimum) from all values in the 4th row.
Deduct 4 (which is the minimum) from all values in the 5th row.*

The resulting row wise reduced cost matrix =
$$\begin{bmatrix} \infty & 10 & 20 & 0 & 1 \\ 13 & \infty & 14 & 2 & 0 \\ 1 & 3 & \infty & 0 & 0 \\ 16 & 3 & 15 & \infty & 0 \\ 12 & 0 & 3 & 12 & \infty \end{bmatrix}$$

Row wise reduction sum = $10 + 2 + 2 + 3 + 4 = 21$

Now apply column reduction for the above matrix:

*Deduct 1 (which is the minimum) from all values in the 1st column.
Deduct 3 (which is the minimum) from all values in the 3rd column.*

The resulting column wise reduced cost matrix (A) =
$$\begin{bmatrix} \infty & 10 & 17 & 0 & 1 \\ 12 & \infty & 11 & 2 & 0 \\ 0 & 3 & \infty & 0 & 2 \\ 15 & 3 & 12 & \infty & 0 \\ 11 & 0 & 0 & 12 & \infty \end{bmatrix}$$

Column wise reduction sum = $1 + 0 + 3 + 0 + 0 = 4$

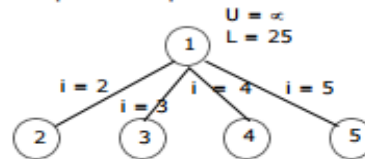
Cumulative reduced sum = row wise reduction + column wise reduction sum.
= $21 + 4 = 25$.

This is the cost of a root i.e., node 1, because this is the initially reduced cost matrix.

The lower bound for node is 25 and upper bound is ∞ .

Starting from node 1, we can next visit 2, 3, 4 and 5 vertices. So, consider to explore the paths (1, 2), (1, 3), (1, 4) and (1, 5).

The tree organization up to this point is as follows:



Variable 'i' indicates the next node to visit.

Step 2:

Consider the path (1, 2):

Change all entries of row 1 and column 2 of A to ∞ and also set A(2, 1) to ∞ .

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 11 & 2 & 0 \\ 0 & \infty & \infty & 0 & 2 \\ 15 & \infty & 12 & \infty & 0 \\ 11 & \infty & 0 & 12 & \infty \end{bmatrix}$$

Apply row and column reduction for the rows and columns whose rows and columns are not completely ∞ .

Then the resultant matrix is

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 11 & 2 & 0 \\ 0 & \infty & \infty & 0 & 2 \\ 15 & \infty & 12 & \infty & 0 \\ 11 & \infty & 0 & 12 & \infty \end{bmatrix}$$

$$\text{Row reduction sum} = 0 + 0 + 0 + 0 = 0$$

$$\text{Column reduction sum} = 0 + 0 + 0 + 0 = 0$$

$$\text{Cumulative reduction } (r) = 0 + 0 = 0$$

$$\text{Therefore, as } \bar{c}(S) = \bar{c}(R) + A(1, 2) + r$$

$$\bar{c}(S) = 25 + 10 + 0 = 35$$

Consider the path (1, 3):

Change all entries of row 1 and column 3 of A to ∞ and also set A(3, 1) to ∞ .

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & \infty & 2 & 0 \\ \infty & 3 & \infty & 0 & 2 \\ 15 & 3 & \infty & \infty & 0 \\ 11 & 0 & \infty & 12 & \infty \end{bmatrix}$$

Apply row and column reduction for the rows and columns whose rows and columns are not completely ∞ .

Then the resultant matrix is

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ & 1 & \infty & \infty & 2 & 0 \\ \infty & 3 & \infty & 0 & 2 \\ 4 & 3 & \infty & \infty & 0 \\ 0 & 0 & \infty & 12 & \infty \end{bmatrix}$$

Row reduction sum = 0

Column reduction sum = 11

Cumulative reduction (r) = 0 + 11 = 11

Therefore, as $\hat{c}(S) = \hat{c}(R) + A(1, 3) + r$

$$\hat{c}(S) = 25 + 17 + 11 = 53$$

Consider the path (1, 4):

Change all entries of row 1 and column 4 of A to ∞ and also set A(4, 1) to ∞ .

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & 11 & \infty & 0 \\ 0 & 3 & \infty & \infty & 2 \\ \infty & 3 & 12 & \infty & 0 \\ 11 & 0 & 0 & \infty & \infty \end{bmatrix}$$

Apply row and column reduction for the rows and columns whose rows and columns are not completely ∞ .

Then the resultant matrix is

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & 11 & \infty & 0 \\ 0 & 3 & \infty & \infty & 2 \\ \infty & 3 & 12 & \infty & 0 \\ 11 & 0 & 0 & \infty & \infty \end{bmatrix}$$

Row reduction sum = 0

Column reduction sum = 0

Cumulative reduction (r) = 0 + 0 = 0

Therefore, as $\bar{c}(S) = \bar{c}(R) + A(1, 4) + r$

$$\bar{c}(S) = 25 + 0 + 0 = 25$$

Consider the path (1, 5):

Change all entries of row 1 and column 5 of A to ∞ and also set A(5, 1) to ∞ .

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & 11 & 2 & \infty \\ 0 & 3 & \infty & 0 & \infty \\ 15 & 3 & 12 & \infty & \infty \\ \infty & 0 & 0 & 12 & \infty \end{bmatrix}$$

Apply row and column reduction for the rows and columns whose rows and columns are not completely ∞ .

Then the resultant matrix is

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 10 & \infty & 9 & 0 & \infty \\ 0 & 3 & \infty & 0 & \infty \\ 12 & 0 & 9 & \infty & \infty \\ \infty & 0 & 0 & 12 & \infty \end{bmatrix}$$

Row reduction sum = 5

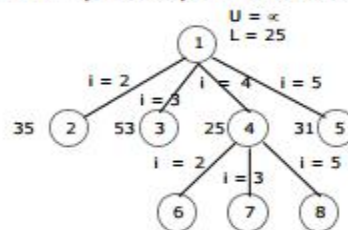
Column reduction sum = 0

Cumulative reduction (r) = 5 + 0 = 5

Therefore, as $\bar{c}(S) = \bar{c}(R) + A(1, 5) + r$

$$\bar{c}(S) = 25 + 1 + 5 = 31$$

The tree organization up to this point is as follows:



The cost of the paths between (1, 2) = 35, (1, 3) = 53, (1, 4) = 25 and (1, 5) = 31. The cost of the path between (1, 4) is minimum. Hence the matrix obtained for path (1, 4) is considered as reduced cost matrix.

$$A = \begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & 11 & \infty & 0 \\ 0 & 3 & \infty & \infty & 2 \\ \infty & 3 & 12 & \infty & 0 \\ 11 & 0 & 0 & \infty & \infty \end{bmatrix}$$

The new possible paths are (4, 2), (4, 3) and (4, 5).

Consider the path (4, 2):

Change all entries of row 4 and column 2 of A to ∞ and also set A(2, 1) to ∞ .

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 11 & \infty & 0 \\ 0 & \infty & \infty & \infty & 2 \\ \infty & \infty & \infty & \infty & \infty \\ 11 & \infty & 0 & \infty & \infty \end{bmatrix}$$

Apply row and column reduction for the rows and columns whose rows and columns are not completely ∞ .

$$\text{Then the resultant matrix is } \begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 11 & \infty & 0 \\ 0 & \infty & \infty & \infty & 2 \\ \infty & \infty & \infty & \infty & \infty \\ 11 & \infty & 0 & \infty & \infty \end{bmatrix}$$

Row reduction sum = 0

Column reduction sum = 0

Cumulative reduction (r) = 0 + 0 = 0

Therefore, as $\bar{c}(S) = \bar{c}(R) + A(4, 2) + r$

$$\bar{c}(S) = 25 + 3 + 0 = 28$$

Consider the path (4, 3):

Change all entries of row 4 and column 3 of A to ∞ and also set A(3, 1) to ∞ .

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & \infty & \infty & 0 \\ \infty & 3 & \infty & \infty & 2 \\ \infty & \infty & \infty & \infty & \infty \\ 11 & 0 & \infty & \infty & \infty \end{bmatrix}$$

Apply row and column reduction for the rows and columns whose rows and columns are not completely ∞ .

Then the resultant matrix is

$$\left[\begin{array}{ccccc|c} \infty & \infty & \infty & \infty & \infty & 0 \\ & 1 & \infty & \infty & \infty & 0 \\ \infty & 1 & \infty & \infty & \infty & 0 \\ \infty & \infty & \infty & \infty & \infty & 0 \\ \infty & 0 & 0 & \infty & \infty & 0 \end{array} \right]$$

Row reduction sum = 2

Column reduction sum = 11

Cumulative reduction (r) = 2 + 11 = 13

Therefore, as $\bar{c}(S) = \bar{c}(R) + A(4, 3) + r$

$$\bar{c}(S) = 25 + 12 + 13 = 50$$

Consider the path (4, 5):

Change all entries of row 4 and column 5 of A to ∞ and also set A(5, 1) to ∞ .

$$\left[\begin{array}{ccccc|c} \infty & \infty & \infty & \infty & \infty & 0 \\ 12 & \infty & 11 & \infty & \infty & 0 \\ 0 & 3 & \infty & \infty & \infty & 0 \\ \infty & \infty & \infty & \infty & \infty & 0 \\ \infty & 0 & 0 & \infty & \infty & 0 \end{array} \right]$$

Apply row and column reduction for the rows and columns whose rows and columns are not completely ∞ .

Then the resultant matrix is

$$\left[\begin{array}{ccccc|c} \infty & \infty & \infty & \infty & \infty & 0 \\ & 1 & \infty & 0 & \infty & 0 \\ 0 & 3 & \infty & \infty & \infty & 0 \\ \infty & \infty & \infty & \infty & \infty & 0 \\ \infty & 0 & 0 & \infty & \infty & 0 \end{array} \right]$$

Row reduction sum = 11

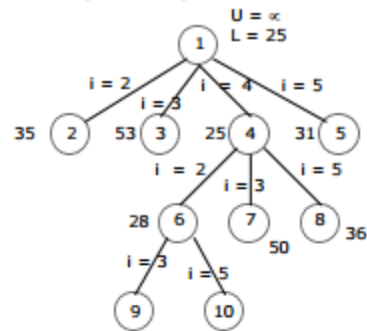
Column reduction sum = 0

Cumulative reduction (r) = 11+0 = 11

Therefore, as $\bar{c}(S) = \bar{c}(R) + A(4, 5) + r$

$$\bar{c}(S) = 25 + 0 + 11 = 36$$

The tree organization up to this point is as follows:



The cost of the paths between $(4, 2) = 28$, $(4, 3) = 50$ and $(4, 5) = 36$. The cost of the path between $(4, 2)$ is minimum. Hence the matrix obtained for path $(4, 2)$ is considered as reduced cost matrix.

$$A = \begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 11 & \infty & 0 \\ 0 & \infty & \infty & \infty & 2 \\ \infty & \infty & \infty & \infty & \infty \\ 11 & \infty & 0 & \infty & \infty \end{bmatrix}$$

The new possible paths are $(2, 3)$ and $(2, 5)$.

Consider the path $(2, 3)$:

Change all entries of row 2 and column 3 of A to ∞ and also set $A(3, 1)$ to ∞ .

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & 2 \\ \infty & \infty & \infty & \infty & \infty \\ 11 & \infty & \infty & \infty & \infty \end{bmatrix}$$

Apply row and column reduction for the rows and columns whose rows and columns are not completely ∞ .

Then the resultant matrix is

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & 0 \\ \infty & \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty & \infty \end{bmatrix}$$

Row reduction sum = 2

Column reduction sum = 11

Cumulative reduction (r) = 2 + 11 = 13

Therefore, as $\bar{c}(S) = \bar{c}(R) + A(2, 3) + r$

$$\bar{c}(S) = 28 + 11 + 13 = 52$$

Consider the path (2, 5):

Change all entries of row 2 and column 5 of A to ∞ and also set A(5, 1) to ∞ .

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 0 & \infty & \infty \end{bmatrix}$$

Apply row and column reduction for the rows and columns whose rows and columns are not completely ∞ .

Then the resultant matrix is

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 0 & \infty & \infty \end{bmatrix}$$

Row reduction sum = 0

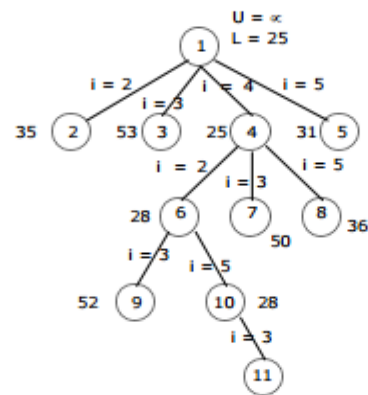
Column reduction sum = 0

Cumulative reduction (r) = 0 + 0 = 0

Therefore, as $\bar{c}(S) = \bar{c}(R) + A(2, 5) + r$

$$\bar{c}(S) = 28 + 0 + 0 = 28$$

The tree organization up to this point is as follows:



The cost of the paths between (2, 3) = 52 and (2, 5) = 28. The cost of the path between (2, 5) is minimum. Hence the matrix obtained for path (2, 5) is considered as reduced cost matrix.

$$A = \begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 0 & \infty & \infty \end{bmatrix}$$

The new possible paths is (5, 3).

Consider the path (5, 3):

Change all entries of row 5 and column 3 of A to ∞ and also set A(3, 1) to ∞ . Apply row and column reduction for the rows and columns whose rows and columns are not completely ∞ .

Then the resultant matrix is

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \end{bmatrix}$$

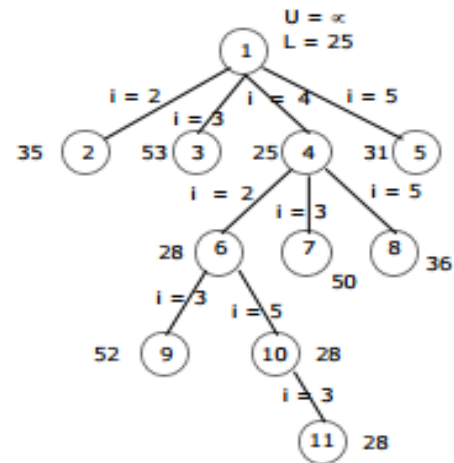
Row reduction sum = 0

Column reduction sum = 0

Cumulative reduction (r) = 0 + 0 = 0

Therefore, as $\bar{c}(S) = \bar{c}(R) + A(5, 3) + r$
 $\bar{c}(S) = 28 + 0 + 0 = 28$

The overall tree organization is as follows:



The path of traveling sale person problem is:

$1 \rightarrow 4 \rightarrow 2 \rightarrow 5 \rightarrow 3 \rightarrow 1$

The minimum cost of the path is: $10 + 6 + 2 + 7 + 3 = 28$.

*P, NP, NP Complete and
NP Hard Problems*

- All the problems can be categorized into two groups.
- The first group consists of the problems that can be solved in polynomial time by **deterministic algorithm** (the algorithms which we have been discussing).
 - Linear search - $O(n)$
 - Binary Search - $O(\log n)$
 - Bubble sort - $O(n^2)$
 - Merge Sort – $O(n \log n)$
 - Matrix multiplication- $O(n^3)$
- Polynomial time: $O(n^2)$, $O(n^3)$, $O(1)$, $O(n \log n)$
- Not in polynomial time: $O(2^n)$, $O(n^n)$, $O(n!)$

- The second group consists of problems that can be solved in polynomial time by non-deterministic algorithms.
- For example:
 - 0/1 Knapsack problem $O(2^{n/2})$
 - Travelling Salesperson problem $O(n^2 2^n)$
 - Graph coloring (2^n).
 - Hamiltonian cycle (2^n).
 - Sum of sub sets (2^n).

- **Definition of P:** Problems that can be solved in polynomial time by using deterministic algorithms. (“p” stand for polynomial.)
- **Definition of NP:** Problems that can be solved in polynomial time by using non-deterministic algorithms.

Non Deterministic Algorithm

- Conceptually a nondeterministic algorithm could run on a deterministic computer with an unlimited number of parallel processors.
- Each time more than one step is possible, new processes are instantly forked to try all of them. When one successfully finishes, that result is returned. Thus the computation is very fast.
- Every nondeterministic algorithm can be turned into a deterministic algorithm, possibly with exponential slow down.
- Consider searching an unordered array. A linear search has $O(n)$ expected time. A nondeterministic search has constant expected time.

Non Deterministic Algorithm

- Up to now, the algorithms that we have been using have the property that the result of every operation is uniquely defined.
- Algorithms with this property are called as deterministic algorithms.
- In a theoretical frame work we can remove this restriction on the outcome of every operation.
- We can allow algorithms to contain operations whose outcomes are not uniquely defined, but are limited to specified sets of possibilities.
- The machine executing such operations is allowed to choose any one of these outcomes.
- Conceptually, a nondeterministic algorithm could run on a deterministic computer with an unlimited number of parallel processors.

- This lead to the concept of a nondeterministic algorithm.
- To specify such algorithms, we introduce three new functions:

Choice(s) : Chooses one of the elements of set S.

Failure () : Signals an unsuccessful completion.

Success() : Signals a successful completion.

- Whenever there is a set of choices that leads to a successful completion, then one such set of choices is always made and the algorithm terminates successfully.
- A nondeterministic algorithm terminates unsuccessfully if and only if there exists no set of choices leading to a success signal.
- The computing time for Choice, Success, and Failure are taken to be $O(1)$.
- A machine capable of executing a nondeterministic algorithm in this way is called a nondeterministic machine.

Example :

- Consider the problem of searching for an element x in a given set of elements $A[1:n]$, $n \geq 1$. We are required to determine an index j such that $a[j]=x$ or $j=0$ if x is not in A . A nondeterministic Algorithm for this is as follows:

```
j:=Choice(1,n);  
if A[j]=x then  
{  
    write( j ); Success();  
}  
else  
{  
    write( 0 ); Failure();  
}
```

- The nondeterministic algorithms terminates unsuccessfully iff there is no set of choices which leads to the successful signal.
- A machine capable of executing a nondeterministic algorithms are known as nondeterministic machines (**does not exist in practice**).

The *Satisfiability* (SAT) Problem

- *Satisfiability* (SAT):
 - Given a Boolean expression on n variables, can we assign values such that the expression is TRUE?
 - Ex: $((x_1 \rightarrow x_2) \vee \neg((\neg x_1 \leftrightarrow x_3) \vee x_4)) \wedge \neg x_2$
 - Seems simple enough, but no known deterministic polynomial time algorithm exists.

Example: CNF satisfiability

- This problem is in *NP*. Nondeterministic algorithm:
- Example:

$$(A \vee \neg B \vee \neg C) \wedge (\neg A \vee B) \wedge (\neg B \vee D \vee F) \wedge (F \vee \neg D)$$

- Truth assignments:

	A	B	C	D	E	F
--	---	---	---	---	---	---

1.	0	1	1	0	1	0
----	---	---	---	---	---	---

2.	1	0	0	0	0	1
----	---	---	---	---	---	---

3.	1	1	0	0	0	1
----	---	---	---	---	---	---

4.	... (how many more?)					
----	----------------------	--	--	--	--	--

There are 2^n (Exponential) values.

Nondeterministic satisfiability

Algorithm Eval(E, n)

// Determine whether the propositional formula E is satisfiable. The variables are $x_1, x_2, \dots, x_n$.

```
{  for i:= 1 to n do  // choose a truth value assignment.  
     $x_i := \text{Choice}(\text{false}, \text{true})$ ;  
    if  $E(x_1, x_2, \dots, x_n)$  then Success( );  
    else Failure( );  
}
```

Time complexity of the above algorithm is $O(n)$.

Note : *above algorithm is a nondeterministic algorithm.*

- **Reducibility**

- A problem Q1 can be reduced to Q2 if any instance of Q1 can be easily reformed as an instance of Q2. If the solution of the problem Q2 provides a solution to the problem Q1, then these are said to be reducible problems.
- Problem L1 can be reduced to L2 if and only if there is a way to solve L1 using deterministic algorithm that solves L2 in polynomial time.
- It is represented as $L1 \alpha L2$
- This definition implies that if we have a polynomial time algorithm for L2, then we can solve L1 in polynomial time.
- Example: $\text{lcm}(m, n) = m * n / \text{gcd}(m, n)$,

Note : All the NP problems can be reducible to satisfiability
(SAT)

NP-Hard and NP-Complete Problems

- Let P denote the set of all decision problems solvable by deterministic algorithm in polynomial time. NP denotes set of problems solvable by nondeterministic algorithms in polynomial time.
- Since, deterministic algorithms are a special case of nondeterministic algorithms or NP is a super set of P , $P \subseteq NP$.

The nondeterministic polynomial time problems can be classified into two classes.

1. NP Hard and 2. NP Complete

NP-Hard:

A problem L is NP-Hard iff satisfiability reduces to L (satisfiability α L).

Ex: Halting Problem, Flow shop scheduling problem.

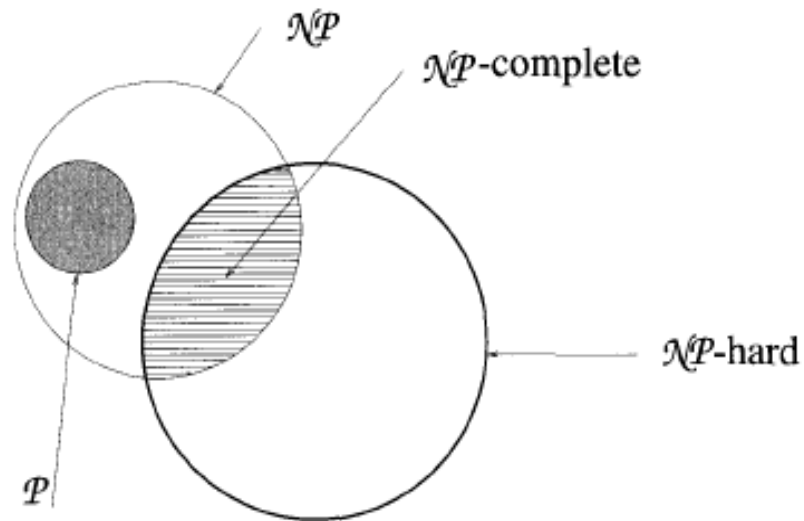
NP-Complete:

A problem L is NP-Complete iff L is NP-Hard and L belongs to NP (nondeterministic polynomial). Examples :

- Boolean satisfiability **problem** (SAT)
- Knapsack **problem**.
- Hamiltonian path **problem**.
- Travelling salesman **problem** (decision version)
- Subgraph isomorphism **problem**.
- Subset sum **problem**.

Note : All the NP- Complete problems are NP-Hard, but some of the NP-Hard problems are not known to be NP-Complete.

Commonly believed relationship among P , NP , NP -Complete, and NP -Hard problems



MODULAR ARITHMETIC

①

1. Modular Arithmetic is useful because it allows the reformulation of way addition, subtraction and multiplication are performed.
2. M A is more useful for the computers that perform parallel computation.

The mod operator is defined as

$$x \bmod y = x - y \left\lfloor \frac{x}{y} \right\rfloor, \text{ if } y \neq 0$$
$$x \bmod 0 = x.$$

→ We denote the set of integers $\{0, 1, \dots, P-1\}$, where P is a prime number, by $GF(P)$ (the Galois field with P elements).

→ The set $GF(P)$ forms a field under the following definition of addition, subtraction, multiplication and division, where $a, b \in GF(P)$.

$$① \quad (a+b) \bmod P = \begin{cases} a+b & \text{if } a+b < P \\ a+b-P & \text{if } a+b \geq P \end{cases}$$

$$② \quad (a-b) \bmod P = \begin{cases} a-b & \text{if } a-b \geq 0 \\ a-b+P & \text{if } a-b < 0. \end{cases}$$

$$③ \quad (ab) \bmod P = r, \text{ ~~such that~~ } r \text{ is the remainder when the product } ab \text{ is divided by } P;$$
$$\Rightarrow ab = qP + r, \text{ where } 0 \leq r < P$$

$$④ \quad (a/b) \bmod P = (a \bar{b}) \bmod P = r, \text{ } r \text{ is the remainder when } a \bar{b} \text{ is divided by } P$$
$$\Rightarrow a \bar{b} = qP + r, \text{ } 0 \leq r < P.$$

→ The factor b^{-1} is the multiplicative inverse of b in $\text{GF}(P)$. (2)

→ For every element b in $\text{GF}(P)$ except zero, there exists a unique element called b^{-1} such that $bb^{-1} \bmod p = 1$.

→ To find $x = b^{-1}$ there must exist an integer x , $0 \leq x < P$ such that $bx = KP + 1$, if $P = 7$,

b :	1	2	3	4	5	6	(element)
b^{-1} :	1	4	5	2	3	6	
K :	0	1	2	1	2	5	

→ Euclid's algorithm is used to calculate inverse of b (i.e., b^{-1}), its time complexity is not more than $O(\log_{10} P)$.

→ The time complexity for addition, subtraction, and multiplication mod P , given the formulas above, is $O(1)$.

Modular Arithmetic transformation technique to work with integers.

Let a & b be integers and suppose that a is represented by the r -tuple $(a_1, \dots, a_r)$, where $a_i = a \bmod P_i$ and b is represented by $(b_1, \dots, b_r)$, where $b_i = b \bmod P_i$. The P_i are prime numbers.

(3)

→ The rules for addition, subtraction, and multiplication are as follows:

$$(a_1 \dots a_r) + (b_1 \dots b_r) = ((a_1 + b_1) \bmod p_1 \dots (a_r + b_r) \bmod p_r)$$

$$(a_1 \dots a_r)(b_1 \dots b_r) = ((a_1 b_1) \bmod p_1, \dots, (a_r b_r) \bmod p_r)$$

For example:

let $p_1 = 3$, $p_2 = 5$, and $p_3 = 7$ and suppose we start with integers 10 and 15.

$$10 = (10 \bmod 3, 10 \bmod 5, 10 \bmod 7) = (1, 0, 3)$$

$$15 = (15 \bmod 3, 15 \bmod 5, 15 \bmod 7) = (0, 0, 1)$$

Then

$$10 + 15 = (25 \bmod 3, 25 \bmod 5, 25 \bmod 7) = (1, 0, 4)$$

$$= (1+0) \bmod 3, (0+0) \bmod 5, (3+1) \bmod 7 = (1, 0, 4)$$

Also

$$15 - 10 = (5 \bmod 3, 5 \bmod 5, 5 \bmod 7) = (2, 0, 5)$$

$$= (0-1) \bmod 3, (0-0) \bmod 5, (1-3) \bmod 7 = (2, 0, 5)$$

Also

$$10 \times 15 = (150 \bmod 3, 150 \bmod 5, 150 \bmod 7) = (0, 0, 3)$$

$$= (1 \times 0 \bmod 3, 0 \times 0 \bmod 5, 3 \times 1 \bmod 7) = (0, 0, 3)$$

Example

Suppose we wish to take 4, 6, and 8

$$\text{Compute } 4 + \overset{6 \times 8}{\cancel{8 \times 6}} = 52$$

$$\text{Let } p_1 = 7, \text{ and } p_2 = 11$$

$$4 = (4 \bmod 7, 4 \bmod 11) = (4, 4)$$

$$6 = (6 \bmod 7, 6 \bmod 11) = (6, 6)$$

$$8 = (8 \bmod 7, 8 \bmod 11) = (1, 8)$$

$$6 \times 8 = ((6 \times 1) \bmod 7, (6 \times 8) \bmod 11) = (6, 4)$$

$$4 + 8 \times 6 = ((4 + 6) \bmod 7, (4 + 4) \bmod 11) = (3, 8)$$

So, we must convert the 2-tuple $(3, 8)$ back to integer notation. Using OneStep Chinese Remainder Algorithm (OneStep CRA) with $a=3$, $b=8$, $p=7$, $q=11$ we get

$$t = a \bmod q = 3 \bmod 11 = 3$$

$$pb = p \bmod q = 7 \bmod 11 = 7$$

$$s = \text{ExEclid}(pb, q) = 8$$

$$u = ((b - t)s) \bmod q = (8 - 3)8 \bmod 11 = 40 \bmod 11 = 7$$

$$\text{return } (u \times p + a) = 7 \times 7 + 3 = \underline{\underline{52}}$$

$$\begin{array}{r} 7 \) \ 15 \ (2 \\ \underline{14} \\ 1 \end{array} \quad \begin{array}{r} 7 \) \ -15 \ (-3) \\ \underline{-21} \\ 6 \end{array}$$

$$|-15| = -21 + 7$$